

FC7xxx WDG User Manual

Rev.0.6

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/12/15	Updated for MCAL V0.4.0 - Optimization plugins
0.6	2023/03/29	Updated for MCAL V0.6.0 - Added support for FC7240

Table of Contents

Revision History	2
Table of Contents	3
Chapter 1 WDG Introduction	5
1.1 Requirement Tracing	5
1.2 Hardware Summary	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure	8
2.3 Macros	9
2.3.1 Macros in Wdg_Version.h	9
2.3.2 Macros in Wdg_LLD.c	9
2.3.3 Macros in Wdg_HLD.h	10
2.3.4 Macros in Wdg_cfg.h	10
2.4 Enums	13
2.4.1 Enumerations in Wdg_HLD.h	13
2.4.2 Enumerations in Wdg_HLD.c	13
2.4.3 Enumerations in Wdg_LLD.h	13
2.5 Typedef	14
2.5.1 Typedefs in Wdg_HLD.h	14
2.6 Structures	14
2.6.1 Wdg_ConfigType	14
2.6.2 Wdg_LLDDConfigType	15
2.7 API Functions	15
2.7.1 Functions in Wdg_Instance0.c	15
2.7.2 Functions in Wdg_Instance1.c	16
2.7.3 Functions in Wdg_Instance2.c	17
2.7.4 Functions in Wdg_HLD.c	18
2.7.5 Functions in Wdg_LLD.c	20
2.7.6 Functions in Wdg_HWA.c	20
2.8 API Sequence Diagram	22
2.8.1 Watchdog Initialization, Setting Trigger Condition and Mode	22
2.8.2 Data Exchange between Watchdog Driver and Hardware	23
Chapter 3 Tresos Configuration Items	24
3.1 Container Inclusion Relation	24
3.2 Containers and Variables	24

3.2.1	IMPLEMENTATION_CONFIG_VARIANT	24
3.2.2	WdgDemEventParameterRefs.....	24
3.2.3	WdgGeneral.....	25
3.2.4	WdgSettingsConfig	28
Chapter 4 Configuration Guides.....		39
4.1	Configuration Item Constraint.....	39
4.2	WDG Usage Common Steps	40
4.3	WDG Configuration Demo	40

Chapter 1 WDG Introduction

1.1 Requirement Tracing

The design of this module follows the specifications of the WDG driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the *AUTOSAR_SWS_WatchdogDriver*.

1.2 Hardware Summary

The Watchdog Timer (WDOG) module is an independent timer that is available for system use. It provides a safety feature to ensure that software is executing as planned and that the CPU is not stuck in an infinite loop or executing unintended code. If the WDOG module is not serviced (refreshed) within a certain period, it resets the MCU.

Features of the WDOG module include:

- 1) Configurable input clock source and independent from the bus clock.
- 2) Programmable 16-bit timeout value composed of high 8 bits value and low 8 bits value in TIMEOUT register.
- 3) Implement an optional fixed 256 clock prescaler to get a larger timeout value.
- 4) Unlock sequence of writing 0x0818_1982 to allow the updates of writeable once registers.
- 5) Software updates must be configured within 128 bus clocks after unlocking.
- 6) Refresh sequence of writing 0x20CF_FC20.
- 7) Programmable 16-bit window value composed of high 8 bits and low 8 bits in WINDOW register.
- 8) Refresh operation earlier than the window value will trigger a reset.
- 9) Optional timeout interrupt to allow post-processing diagnostics after a counter timeout occurs.
- 10) Delay of 128, 256, 512, or 1024 bus clocks to generate a reset after the interrupt vector fetched.
- 11) Registers related to watchdog configuration are write-once-after-reset to ensure that the configuration cannot be modified by mistake.

WDOG0 and WDOG1 are available on both FC7300 and FC7300, and WDOG2 is only available on FC7300.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 **SWS_Wdg_00093**

Description	If the watchdog hardware requires an activation code which can be configured or changed, the Wdg Driver shall handle the activation code internally. In this case, the Wdg Driver shall pass the correct activation code to the watchdog hardware and the watchdog hardware in turn shall update the Wdg module's internal variable where the next expected access code is stored.
Rejection Reason	The activation code used by our applicable watchdog is fixed.

Rejected Requirement 2 **SWS_Wdg_00094**

Description	If the watchdog hardware requires an activation code which can be configured or changed, the trigger cycle of the Wdg Driver shall be defined with a value so that updating the activation code by the watchdog hardware can be guaranteed (see Figure 3).
Rejection Reason	The activation code used by our applicable watchdog is fixed.

Rejected Requirement 3 **SWS_Wdg_00095**

Description	If the watchdog hardware requires an activation code which can be configured or changed and the initial activation code can be configured, the activation code shall be provided in the Wdg Driver's configuration set. If the activation code is fixed for a particular hardware the above requirement can be ignored.
Rejection Reason	The activation code used by our applicable watchdog is fixed.

Rejected Requirement 4 **SWS_Wdg_00055**

Description	The WDG module for an external watchdog driver shall have source code that is independent of the microcontroller platform.
Rejection Reason	External module is customer dependent and is not developed, and therefore not in scope. External assumption - "An external device, acting as supervisor of the operations, must provide a watchdog to cover common cause of failures of Leopard_1M for SIL3 applications". The implementation for WDG external must be supported by a dedicated WDG module

Rejected Requirement 5 **SWS_Wdg_00076**

Description	To access the external watchdog hardware, the corresponding WDG module instance shall use the functionality and API of the corresponding handler or driver, e.g. the SPI handler or DIO driver.
Rejection Reason	External module is customer dependent and is not developed, and therefore not in scope. External assumption - "An external device, acting as supervisor of the operations, must provide a watchdog to cover common cause of failures of Leopard_1M for SIL3 applications". The implementation for WDG external must be supported by a dedicated WDG module

Rejected Requirement 6 SWS_Wdg_00077

Description	A WDG module for an external watchdog shall satisfy the same functional requirements and offer the same functional scope as a WDG module for an internal watchdog. Hence their respective APIs are semantically identical.
Rejection Reason	External module is customer dependent and is not developed, and therefore not in scope. External assumption - "An external device, acting as supervisor of the operations, must provide a watchdog to cover common cause of failures of Leopard_1M for SIL3 applications". The implementation for WDG external must be supported by a dedicated WDG module

Rejected Requirement 7 SWS_Wdg_00078

Description	The WDG module shall add all parameters required for accessing the external watchdog hardware, e.g. the used SPI channel or DIO port, to the module's published parameters and to the module's configuration parameters.
Rejection Reason	External module is customer dependent and is not developed, and therefore not in scope. External assumption - "An external device, acting as supervisor of the operations, must provide a watchdog to cover common cause of failures of Leopard_1M for SIL3 applications". The implementation for WDG external must be supported by a dedicated WDG module

Rejected Requirement 8 SWS_Wdg_000172

Description	If more than one watchdog driver instance exists on an ECU (namely an external and an internal one) the API names and instance specific type names specified in this chapter shall be made unique by expansion according to SRS_BSW_00347.
Rejection Reason	External module is customer dependent and is not developed, and therefore not in scope. External assumption - "An external device, acting as supervisor of the operations, must provide a watchdog to cover common cause of failures of Leopard_1M for SIL3 applications". The implementation for WDG external must be supported by a dedicated WDG module.

Rejected Requirement 9 ECUC_Wdg_00112

Description	Container Name - WdgExternalConfiguration - Description - Configuration items for an external watchdog hardware - Configuration Parameters -
Rejection Reason	External module is customer dependant and is not developed, and therefore not in scope. The implementation for WDG external must be supported by a dedicated Wdg module.

Rejected Requirement 10 ECUC_Wdg_00113

Description	SWS Item	ECUC_Wdg_00113
	Name	WdgExternalContainerRef
	Parent Container	WdgExternalConfiguration
	Description	Reference to either a DioChannelGroup container in case the hardware watchdog is connected via DIO pins or a SpiSequenceConfiguration container in case the watchdog hardware is accessed via SPI

	Multiplicity	0..1		
	Type	Choice reference to [DioChannelGroup , SpiSequence] -		
	Post-Build Variant Multiplicity	true		
	Post-Build Variant Value	true		
	Multiplicity Configuration Class	Pre-compile time	X	VARIANT-PRE-COMPILE
		Link time	X	VARIANT-LINK-TIME
		Post-build time	X	VARIANT-POST-BUILD
	Value Configuration Class	Pre-compile time	X	VARIANT-PRE-COMPILE
		Link time	–	VARIANT-LINK-TIME
		Post-build time	X	VARIANT-POST-BUILD
	Scope / Dependency	scope: local dependency: See DIO resp. SPI SWS		
Rejection Reason	External module is customer dependant and is not developed, and therefore not in scope. The implementation for WDG external must be supported by a dedicated Wdg module.			

2.2 File Structure

Figure 1. WDG instance0

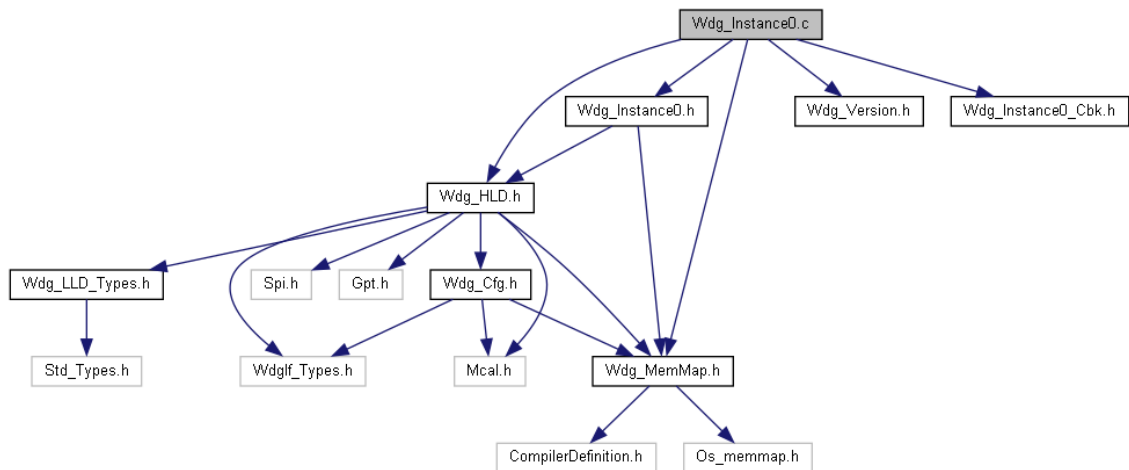


Figure 2. WDG instance1

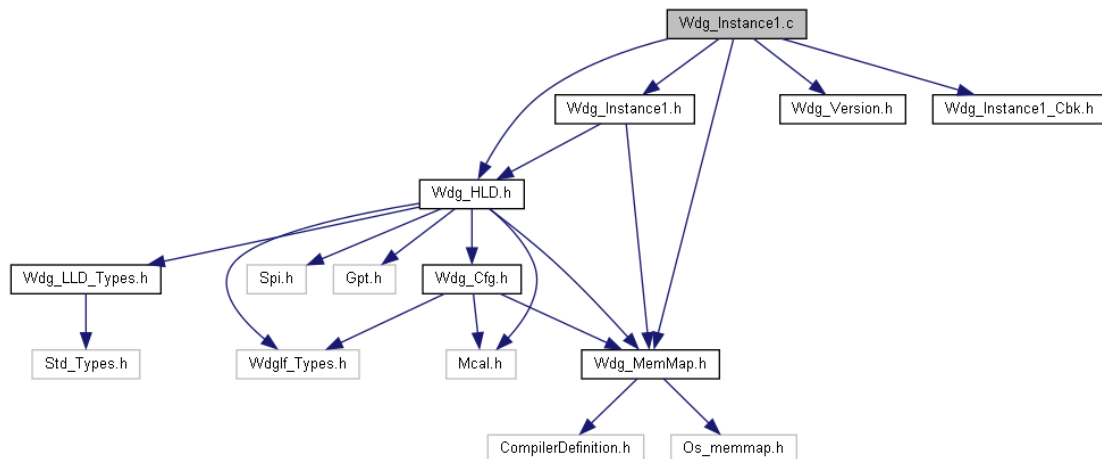
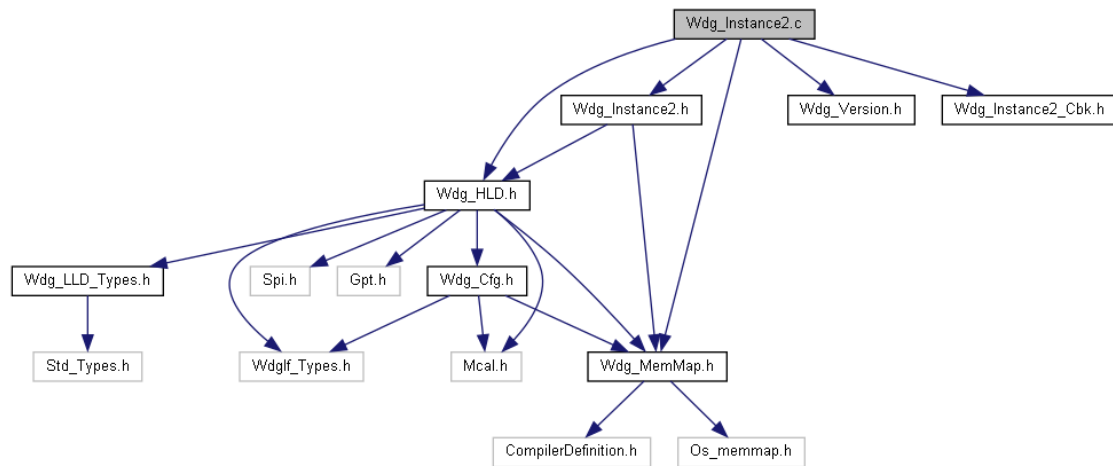


Figure 3. WDG instance2

2.3 Macros

2.3.1 Macros in Wdg_Version.h

- #define **WDG_AR_RELEASE_MAJOR_VERSION** 4
- #define **WDG_AR_RELEASE_MINOR_VERSION** 6
- #define **WDG_AR_RELEASE_REVISION_VERSION** 0
- #define **WDG_SW_MAJOR_VERSION** 0
- #define **WDG_SW_MINOR_VERSION** 6
- #define **WDG_SW_PATCH_VERSION** 0
- #define **WDG_VENDOR_ID** 174
- #define **WDG_MODULE_ID** 102

2.3.2 Macros in Wdg_LLD.c

- #define **WDOG_UNLOCK_CMD** (uint32)(0x08181982U)
Used to "Unlock sequence of writing 0x0818_1982 to allow the updates of writeable once registers".
- #define **WDOG_REFRESH_CMD** (uint32)(0x20CFFC20U)
Used to "Refresh sequence of writing 0x20CF_FC20".
- #define **WDG_PRESCALER_SHIFT_VALUE** (uint16)(8U)
Displacement parameters for participating in frequency division.
- #define **WDG_RECONFIGURATION_TIMEOUT** (uint32)(65535U)
Parameter base value for configuring timeout detection.

2.3.3 Macros in Wdg_HLD.h

- **#define WDG_E_DRIVER_STATE** ((uint8)0x10U)
API service used in wrong context (e.g. driver not initialized)
- **#define WDG_E_PARAM_MODE** ((uint8)0x11U)
API service called with wrong/inconsistent parameter(s)
- **#define WDG_E_PARAM_CONFIG** ((uint8)0x12U)
API service called with wrong/inconsistent parameter(s)
- **#define WDG_E_PARAM_TIMEOUT** ((uint8)0x13U)
API service called with wrong/inconsistent parameter(s)
- **#define WDG_E_PARAM_POINTER** ((uint8)0x14U)
API is called with wrong pointer value (e.g. NULL pointer)
- **#define WDG_E_INIT_FAILED** ((uint8)0x15U)
Invalid configuration set selection
- **#define WDG_INIT_ID** ((uint8)0x00U)
API service ID for Wdg_Instancex_Init function (x=0,1)
- **#define WDG_SETMODE_ID** ((uint8)0x01U)
API service ID for Wdg_Instancex_SetMode function (x=0,1)
- **#define WDG_SETTRIGGERCONDITION_ID** ((uint8)0x02U)
API service ID for Wdg_Instancex_SetTriggerCondition function (x=0,1)
- **#define WDG_TRIGGER_ID** ((uint8)0x03U)
API service ID for Wdg_Cbk_GptNotification0 function
- **#define WDG_GETVERSION_ID** ((uint8)0x04U)
API service ID for Wdg_Instancex_GetVersionInfo function (x=0,1)

2.3.4 Macros in Wdg_cfg.h

- **#define WDG_INSTANCE0** (STD_ON)
This constant specifies if the instance of the WDG is selected or not.
- **#define WDG_INSTANCE1** (STD_ON)
This constant specifies if the instance of the WDG is selected or not.
- **#define WDG_INSTANCE2** (STD_OFF)
This constant specifies if the instance of the WDG is selected or not.

- **#define WDG_INTERNAL_MODULE (0U)**
This define indicates that the watchdog is internal.
- **#define WDG_EXTERNAL_MODULE_SPI (1U)**
This define indicates that the watchdog is externally connected on SPI.
- **#define WDG_EXTERNAL_MODULE_DIO (2U)**
This define indicates that the watchdog is externally connected on DIO.
- **#define WDG_NO_OF_INSTANCES (3U)**
This define indicates the number of HW IPs available on the Platform.
- **#define WDG_TYPE (WDG_INTERNAL_MODULE)**
This variable will indicate which type of driver is in use.
- **#define WDG_DISABLE_DEM_REPORT_ERROR_STATUS (STD_ON)**
Compile switch to enable/disable DEM for this module.
- **#define WDG_DEV_ERROR_DETECT (STD_ON)**
Compile switch to enable/disable development error detection for this module.
- **#define WDG_DISABLE_ALLOWED (STD_OFF)**
Compile switch to allow/forbid disabling the watchdog driver during runtime.
- **#define WDG_VERSION_INFO_API (STD_ON)**
Compile switch to enable/disable the version information.
- **WdgConf_WdgGeneral_WdgIndex_U8 ((uint8)0)**
This variable will indicate the index of instance 0.
- **WdgConf_WdgGeneral_WdgIndex1_U8 ((uint8)1)**
This variable will indicate the index of instance 1.
- **WdgConf_WdgGeneral_WdgIndex2_U8 ((uint8)2)**
This variable will indicate the index of instance 2. (only available on FC7300)
- **#define WDG_INSTANCE0_INITIAL_TIMEOUT_U32 ((uint16)30)**
These variables will indicate the Wdg Initial Timeout parameters in milliseconds for Wdg module 0
- **#define WDG_INSTANCE1_INITIAL_TIMEOUT_U32 ((uint16)30)**
These variables will indicate the Wdg Initial Timeout parameters in milliseconds for Wdg module 1

- **#define WDG_INSTANCE1_INITIAL_TIMEOUT_U32 ((uint16)30)**
These variables will indicate the Wdg Initial Timeout parameters in milliseconds for Wdg module 2
- **#define WDG_INTIAL_TIMEOUT**
{WDG_INSTANCE0_INITIAL_TIMEOUT_U32,WDG_INSTANCE1_INITIAL_TIMEOUT_U32,WDG_INSTANCE2_INITIAL_TIMEOUT_U32}
Wdg Initial Timeout parameters of s_WdgInitialTimeout
- **#define WDG_INSTANCE0_MAX_TIMEOUT_VALUE ((uint16)100)**
These variables will indicate the Wdg Max Timeout parameters in milliseconds for Wdg module 0
- **#define WDG_INSTANCE1_MAX_TIMEOUT_VALUE ((uint16)100)**
These variables will indicate the Wdg Max Timeout parameters in milliseconds for Wdg module 1
- **#define WDG_INSTANCE2_MAX_TIMEOUT_VALUE ((uint16)100)**
These variables will indicate the Wdg Max Timeout parameters in milliseconds for Wdg module 2
- **#define WDG_MAX_TIMEOUT**
{WDG_INSTANCE0_MAX_TIMEOUT_VALUE,WDG_INSTANCE1_MAX_TIMEOUT_VALUE,WDG_INSTANCE2_MAX_TIMEOUT_VALUE}
Wdg max Timeout parameters of s_WdgMaxTimeout
- **#define WDG_MAX_VALUE_TYPE16 (65535U)**
This macro will indicate the max value of 16bits integer type.
- **#define WDOG_MIN_VALUE_TIMEOUT_U32 (0U)**
This macro will indicate min value of timeout.
- **#define WDG_DEM_REPORT_DISABLE_REJECT (STD_OFF)**
This macro indicates if enable the DEM function to report DISABLE REJECT error.
- **#define WDG_DEM_REPORT_MODE_FAIL (STD_OFF)**
This macro indicates if enable the DEM function to report a MODE FAIL error.
- **#define WDG_ISR0_USED**
This variable will indicate if the ISR0 is used.
- **#define WDG_ISR1_USED**
This variable will indicate if the ISR1 is used.
- **#define WDG_ISR2_USED**
This variable will indicate if the ISR2 is used.

- #define **WDG_ROM**
This variable will indicate RAM/ROM execution.
- #define **WDG_PRECOMPILE_SUPPORT** (STD_ON)
Defines the use of Pre compile (PC) support.
- #define **WDG_MULTICORE_ENABLED** (STD_OFF)
Defines the multi-core support.

2.4 Enums

2.4.1 Enumerations in Wdg_HLD.h

2.4.1.1 Wdg_HWL_InstanceType

Enumeration	Wdg_HWL_InstanceType	
Description	Hardware WDG instance.	
Values	Value	Description
	WDG_HWL_INSTANCE0 = 0	Watch dog instance 0.
	WDG_HWL_INSTANCE1 = 1	Watch dog instance 1.
	WDG_HWL_INSTANCE1 = 2	Watch dog instance 2.

2.4.2 Enumerations in Wdg_HLD.c

2.4.2.1 Wdg_StatusType

Enumeration	Wdg_StatusType	
Description	This enumerated type will contain the watchdog driver's possible states.	
Values	Value	Description
	WDG_UNINIT = 0	The watchdog driver is not uninitialized.
	WDG_IDLE = 1	The watchdog driver is currently idle.
	WDG_BUSY = 2	The watchdog driver is currently busy.
	WDG_TRIGGER_ID = 3	The watchdog driver is currently initializing.

2.4.3 Enumerations in Wdg_LLD.h

2.4.3.1 WDOG_ClkSelType

Enumeration	WDOG_ClkSelType	
Description	WDOG clock source select, mapping with WDOG_CS[CLK_SEL] bit field definition.	
Values	Value	Description
	WDOG_BUS_CLK = 0	Bus clock.
	WDOG_AON_CLK = 1	AON clock, if choose this clock need to configure AON clock in SMISC setting.
	WDOG_FOSC_CLK = 2	FOSC clock.
	WDOG_SIRC_CLK = 3	SIRC clock.

2.4.3.2 WDOG_TestType

Enumeration	WDOG_TestType	
Description	WDOG test mode, mapping with WDOG_CS[TST] bit filed definition.	
Values	Value	Description
	WDOG_TST_DISABLED = 0	Test mode disabled.
	WDOG_USER_MODE = 1	User mode enabled. (Test mode disabled.)
	WDOG_TST_MODE_LOW = 2	Test mode enabled, only the low byte is used.
	WDOG_TST_MODE_HIGH = 3	Test mode enabled, only the high byte is used.

2.4.3.3 WDOG_TimeoutReactionType

Enumeration	WDOG_TimeoutReactionType	
Description	WDOG delay time to generate the reset.	
Values	Value	Description
	WDOG_REACTION_INT_128_CYCLE = 0	Need to enable the timeout interrupt, and the delay to generate the reset is configured to 128 bus clock cycles.
	WDOG_REACTION_INT_256_CYCLE = 1	Need to enable the timeout interrupt, and the delay to generate the reset is configured to 256 bus clock cycles.
	WDOG_REACTION_INT_512_CYCLE = 2	Need to enable the timeout interrupt, and the delay to generate the reset is configured to 512 bus clock cycles.
	WDOG_REACTION_INT_1024_CYCLE = 3	Need to enable the timeout interrupt, and the delay to generate the reset is configured to 1024 bus clock cycles.

2.5 Typedef

2.5.1 Typedefs in Wdg_HLD.h

- typedef void (* Wdg_CallbackPtrType) (void);
Pointer to callback notification in the Wdg_Irq routine.

2.6 Structures

2.6.1 Wdg_ConfigType

Structure	Wdg_ConfigType
Description	Used for pointers to structures holding configuration data provided to the WDG module initialization routine for configuration of the module and watchdog hardware.
Diagram	N/A
Data Fields	- WdgIf_ModeType eWdgDefaultSetMode The default mode for initializing phase. - Wdg_HWL_InstanceType eWdgInstance The instance ID. - Gpt_ChannelType u8WdgTimerChannel GPT Channel configured, used for server the watchdog trigger routine. - uint32 u32WdgTriggerChannelFrequency The frequency of the configured timer channel. - Wdg_HWL_ConfigType* pWdgModeSettings [3]

	Pointer to Watchdog Specific implementation details in each mode. Wdg_CallbackPtrType pWdgCallbackNotification Pointer to callback notification.
--	---

2.6.2 Wdg_LLDDConfigType

Structure	Wdg_LLDDConfigType
Description	Contains the information related to the hardware setup for watchdog.
Diagram	N/A
Data Fields	- uint32 u32WdgLLDConfig WDOG configuration for CS register. - uint32 u32WdgLLDTimeout WDOG timeout counter value for TIMEOUT register. - uint32 u32WdgLLDWindow WDOG window counter value for WINDOW register. - boolean bWdgLLDPrescalerEn WDOG prescaler enabled. - uint32 u32WdgLLDInternalTimerClock WDOG internal clock frequency.

2.7 API Functions

2.7.1 Functions in Wdg_Instance0.c

2.7.1.1 void Wdg_Instance0_Init(const Wdg_ConfigType* ConfigPtr)

Function	void Wdg_Instance0_Init(const Wdg_ConfigType* ConfigPtr)	
Description	WDG initialization.	
Diagram	<pre> graph LR Wdg_Instance0_Init --> Wdg_HLD_Init Wdg_HLD_Init --> Wdg_HLD_StartTimerCount Wdg_HLD_Init --> Wdg_HLD_ValidateModeConfiguration Wdg_HLD_StartTimerCount --> Wdg_HWL_Init Wdg_HLD_StartTimerCount --> Wdg_HWL_SetMode Wdg_HLD_ValidateModeConfiguration --> Wdg_HLD_DemEventControl Wdg_HWL_Init --> Wdg_LLDD_Init Wdg_HWL_SetMode --> Wdg_LLDD_Init Wdg_LLDD_Init --> Wdg_LLDD_Unlock Wdg_LLDD_Init --> WDOG_HWA_GetCs Wdg_LLDD_Init --> WDOG_HWA_SetCs Wdg_LLDD_Init --> WDOG_HWA_SetTimeout Wdg_LLDD_Init --> WDOG_HWA_SetWindow Wdg_LLDD_Unlock --> WDOG_HWA_SetCounter </pre>	
Parameters	Parameter	Description
	ConfigPtr	Pointer to configuration set
Returns	N/A	

2.7.1.2 Std_ReturnType Wdg_Instance0_SetMode(WdgIf_ModeType Mode)

Function	Std_ReturnType Wdg_Instance0_SetMode(WdgIf_ModeType Mode)	
Description	Switch the watchdog mode according to the input parameter Mode.	
Diagram	<pre> graph LR Wdg_Instance0_SetMode --> Wdg_HLD_SetMode Wdg_HLD_SetMode --> Wdg_HLD_StartTimerCount Wdg_HLD_SetMode --> Wdg_HLD_ValidateConfigUpdate Wdg_HLD_StartTimerCount --> Wdg_HWL_Init Wdg_HLD_StartTimerCount --> Wdg_HWL_SetMode Wdg_HLD_ValidateConfigUpdate --> Wdg_HLD_ValidateModeConfiguration Wdg_HLD_ValidateConfigUpdate --> Wdg_HWL_ValidateConfigUpdate Wdg_HLD_ValidateModeConfiguration --> Wdg_HLD_DemEventControl Wdg_HWL_ValidateConfigUpdate --> Wdg_LLDD_ValidateConfigUpdate Wdg_LLDD_ValidateConfigUpdate --> Wdg_LLDD_Unlock Wdg_LLDD_ValidateConfigUpdate --> WDOG_HWA_SetCs Wdg_LLDD_ValidateConfigUpdate --> WDOG_HWA_SetTimeout Wdg_LLDD_ValidateConfigUpdate --> WDOG_HWA_SetWindow Wdg_LLDD_Unlock --> WDOG_HWA_SetCounter </pre>	

Parameters	Parameter	Description
	Mode	Mode to be configured
Returns	Std_ReturnType:	
	E_OK	Mode switch executed completely and successfully.
	E_NOT_OK	The mode switch encountered errors.

2.7.1.3 void Wdg_Instance0_SetTriggerCondition(uint16 timeout)

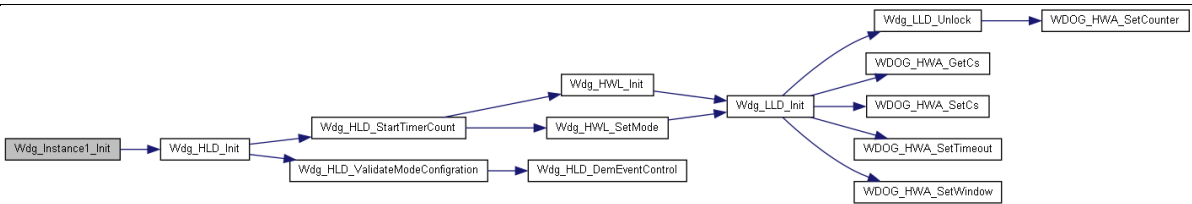
Function	void Wdg_Instance0_SetTriggerCondition(uint16 timeout)	
Description	Reset the watchdog timeout counter according to the parameter timeout.	
Diagram	 <pre> graph LR A[Wdg_Instance0_SetTriggerCondition] --> B[Wdg_HLD_SetTriggerCondition] </pre>	
Parameters	Parameter	Description
	timeout	Value (milliseconds) for setting the trigger counter.
Returns	N/A	

2.7.1.4 void Wdg_Instance0_GetVersionInfo(Std_VersionInfoType * versioninfo)

Function	void Wdg_Instance0_GetVersionInfo(Std_VersionInfoType * versioninfo)	
Description	Get WDG module version information.	
Diagram	 <pre> graph LR A[Wdg_Instance0_GetVersionInfo] --> B[Wdg_HLD_GetVersionInfo] </pre>	
Parameters	Parameter	Description
	versioninfo	Pointer to version structure.
Returns	N/A	

2.7.2 Functions in Wdg_Instance1.c

2.7.2.1 void Wdg_Instance1_Init(const Wdg_ConfigType* ConfigPtr)

Function	void Wdg_Instance1_Init(const Wdg_ConfigType* ConfigPtr)	
Description	WDG initialization.	
Diagram	 <pre> graph LR A[Wdg_Instance1_Init] --> B[Wdg_HLD_Init] B --> C[Wdg_HLD_StartTimerCount] B --> D[Wdg_HLD_ValidateModeConfiguration] C --> E[Wdg_HWL_SetMode] D --> E E --> F[Wdg_LLD_Init] F --> G[Wdg_LLD_Unlock] F --> H[WDOG_HWA_GetCs] F --> I[WDOG_HWA_SetCs] F --> J[WDOG_HWA_SetTimeout] F --> K[WDOG_HWA_SetWindow] G --> L[WDOG_HWA_SetCounter] </pre>	
Parameters	Parameter	Description
	ConfigPtr	Pointer to configuration set.
Returns	N/A	

2.7.2.2 Std_ReturnType Wdg_Instance1_SetMode(WdgIf_ModeType Mode)

Function	Std_ReturnType Wdg_Instance1_SetMode(WdgIf_ModeType Mode)	
Description	Switch the watchdog mode according to the input parameter Mode.	

Diagram		
Parameters	Parameter	Description
	Mode	Mode to be configured.
Returns	Std_ReturnType: E_OK Mode switch executed completely and successfully. E_NOT_OK The mode switch encountered errors.	

2.7.2.3 void Wdg_Instance1_SetTriggerCondition(uint16 timeout)

Function	void Wdg_Instance1_SetTriggerCondition(uint16 timeout)	
Description	Reset the watchdog timeout counter according to the parameter timeout.	
Diagram		
Parameters	Parameter	Description
	timeout	Value (milliseconds) for setting the trigger counter.
Returns	N/A	

2.7.2.4 void Wdg_Instance1_GetVersionInfo(Std_VersionInfoType * versioninfo)

Function	void Wdg_Instance1_GetVersionInfo(Std_VersionInfoType * versioninfo)	
Description	Get WDG module version information.	
Diagram		
Parameters	Parameter	Description
	versioninfo	Pointer to version structure
Returns	N/A	

2.7.3 Functions in Wdg_Instance2.c

2.7.3.1 void Wdg_Instance2_Init(const Wdg_ConfigType* ConfigPtr)

Function	void Wdg_Instance2_Init(const Wdg_ConfigType* ConfigPtr)	
Description	WDG initialization.	
Diagram		
Parameters	Parameter	Description
	ConfigPtr	Pointer to configuration set.
Returns	N/A	

2.7.3.2 Std_ReturnType Wdg_Instance1_SetMode(WdgIf_ModeType Mode)

Function	Std_ReturnType Wdg_Instance1_SetMode(WdgIf_ModeType Mode)	
Description	Switch the watchdog mode according to the input parameter Mode.	
Diagram		
Parameters	Parameter	Description
	Mode	Mode to be configured.
Returns	Std_ReturnType: E_OK Mode switch executed completely and successfully. E_NOT_OK The mode switch encountered errors.	

2.7.3.3 void Wdg_Instance1_SetTriggerCondition(uint16 timeout)

Function	void Wdg_Instance1_SetTriggerCondition(uint16 timeout)	
Description	Reset the watchdog timeout counter according to the parameter timeout.	
Diagram		
Parameters	Parameter	Description
	timeout	Value (milliseconds) for setting the trigger counter.
Returns	N/A	

2.7.3.4 void Wdg_Instance1_GetVersionInfo(Std_VersionInfoType * versioninfo)

Function	void Wdg_Instance1_GetVersionInfo(Std_VersionInfoType * versioninfo)	
Description	Get WDG module version information.	
Diagram		
Parameters	Parameter	Description
	versioninfo	Pointer to version structure
Returns	N/A	

2.7.4 Functions in Wdg_HLD.c

2.7.4.1 void Wdg_HLD_Init(const Wdg_ConfigType* pConfigPtr, const Wdg_HWL_InstanceType eInstance)

Function	void Wdg_HLD_Init(const Wdg_ConfigType* pConfigPtr, const Wdg_HWL_InstanceType eInstance)	
Description	This function initializes the WDG module.	
Diagram		
Parameters	Parameter	Description

	pConfigPtr	Pointer to configuration set.
	eInstance	Hardware WDG instance.
Returns	N/A	

2.7.4.2 Std_ReturnType Wdg_HLD_SetMode(WdgIf_ModeType Mode, const Wdg_HWL_InstanceType eInstance)

Function	Std_ReturnType Wdg_HLD_SetMode(WdgIf_ModeType Mode, const Wdg_HWL_InstanceType eInstance)	
Description	Switches the watchdog into the mode Mode.	
Diagram	<pre> graph LR Wdg_HLD_SetMode --> Wdg_HLD_StartTimerCount Wdg_HLD_SetMode --> Wdg_HLD_ValidateConfigUpdate Wdg_HLD_StartTimerCount --> Wdg_HWL_SetMode Wdg_HLD_ValidateConfigUpdate --> Wdg_HWL_ValidateConfigUpdate Wdg_HWL_SetMode --> Wdg_HWL_Init Wdg_HWL_ValidateConfigUpdate --> Wdg_LLD_ValidateConfigUpdate Wdg_HWL_Init --> Wdg_LLD_Init Wdg_LLD_Init --> Wdg_LLD_Unlock Wdg_LLD_Init --> WDOG_HWA_SetCs Wdg_LLD_Init --> WDOG_HWA_SetTimeout Wdg_LLD_Init --> WDOG_HWA_SetWindow Wdg_LLD_Init --> WDOG_HWA_GetCs Wdg_LLD_Unlock --> WDOG_HWA_SetCounter Wdg_LLD_ValidateConfigUpdate --> WDOG_HWA_GetCs </pre>	
Parameters	Parameter	Description
	Mode	Mode to be configured.
	eInstance	Hardware WDG instance.
Returns	Std_ReturnType: E_OK Mode switch executed completely and successfully. E_NOT_OK The mode switch encountered errors.	

2.7.4.3 void Wdg_HLD_SetTriggerCondition(uint16 u16Timeout, const Wdg_HWL_InstanceType eInstance)

Function	void Wdg_HLD_SetTriggerCondition(uint16 u16Timeout, const Wdg_HWL_InstanceType eInstance)	
Description	Reset the watchdog timeout counter according to the parameter timeout.	
Diagram	<pre> graph LR Wdg_Instance0_SetTriggerCondition --> Wdg_HLD_SetTriggerCondition Wdg_Instance1_SetTriggerCondition --> Wdg_HLD_SetTriggerCondition </pre>	
Parameters	Parameter	Description
	u16Timeout	Value (milliseconds) for setting the trigger counter.
	eInstance	Hardware WDG instance.
Returns	N/A	

2.7.4.4 void Wdg_ProcessInterrupt(const Wdg_HWL_InstanceType eInstance)

Function	void Wdg_ProcessInterrupt(const Wdg_HWL_InstanceType eInstance)	
Description	The Wdg_Isr is used to detect missed trigger of the hardware.	
Diagram	N/A	
Parameters	Parameter	Description
	eInstance	Hardware WDG instance.
Returns	N/A	

2.7.5 Functions in Wdg_LLD.c

2.7.5.1 uint32 Wdg_LLD_Init(const Wdg_LLDConfigType* pWdgLLDConfigPtr, const Wdg_HWL_InstanceType eInstance)

Function	uint32 Wdg_LLD_Init(const Wdg_LLDConfigType* pWdgLLDConfigPtr, const Wdg_HWL_InstanceType eInstance)	
Description	Initialize the hardware of WDG module.	
Diagram	<pre> graph LR Wdg_LLD_Init[Wdg_LLD_Init] --> Wdg_LLD_Unlock[Wdg_LLD_Unlock] Wdg_LLD_Init --> WDOG_HWA_GetCs[WDOG_HWA_GetCs] Wdg_LLD_Init --> WDOG_HWA_SetCs[WDOG_HWA_SetCs] Wdg_LLD_Init --> WDOG_HWA_SetTimeout[WDOG_HWA_SetTimeout] Wdg_LLD_Init --> WDOG_HWA_SetWindow[WDOG_HWA_SetWindow] Wdg_LLD_Unlock --> WDOG_HWA_SetCounter[WDOG_HWA_SetCounter] </pre>	
Parameters	Parameter	Description
	pWdgLLDConfigPtr	Pointer to configuration set.
	eInstance	Hardware WDG instance.
Returns	N/A	

2.7.5.2 void Wdg_LLD_Trigger(const Wdg_HWL_InstanceType eInstance)

Function	void Wdg_LLD_Trigger(const Wdg_HWL_InstanceType eInstance)	
Description	This function triggers the watchdog hardware. Writes the trigger sequence on the hardware	
Diagram	<pre> graph LR Wdg_LLD_Trigger[Wdg_LLD_Trigger] --> WDOG_HWA_SetCounter[WDOG_HWA_SetCounter] </pre>	
Parameters	Parameter	Description
	eInstance	Hardware WDG instance.
Returns	N/A	

2.7.5.3 Std_ReturnType Wdg_LLD_ValidateConfigUpdate(const Wdg_HWL_InstanceType eInstance)

Function	Std_ReturnType Wdg_LLD_ValidateConfigUpdate(const Wdg_HWL_InstanceType eInstance)	
Description	The function checks if the WDG configuration update is allowed by hardware	
Diagram	<pre> graph LR Wdg_LLD_ValidateConfigUpdate[Wdg_LLD_ValidateConfigUpdate] --> WDOG_HWA_GetCs[WDOG_HWA_GetCs] </pre>	
Parameters	Parameter	Description
	eInstance	Hardware WDG instance.
Returns	Std_ReturnType: E_OK Mode switch executed completely and successfully. E_NOT_OK The mode switch encountered errors.	

2.7.6 Functions in Wdg_HWA.c

2.7.6.1 uint32 WDOG_HWA_GetCs(WDOG_Type * pWdog)

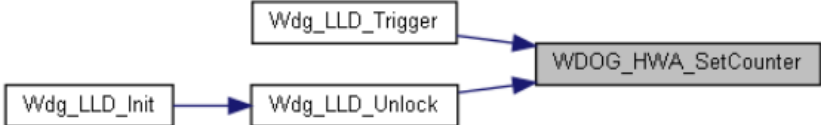
Function	uint32 WDOG_HWA_GetCs(WDOG_Type * pWdog)
-----------------	--

Description	This function Get CS register value, for WDOG working mode configuration.	
Diagram	 <pre> graph LR Wdg_LLD_Init --> WDOG_HWA_GetCs Wdg_LLD_ValidateConfigUpdate --> WDOG_HWA_GetCs </pre>	
Parameters	Parameter	Description
	pWdog	Point to WDOG instance base register address.
Returns	Uint32: uint32 CS register value	

2.7.6.2 boolean WDOG_HWA_GetUnlockStatus(WDOG_Type * pWdog)

Function	boolean WDOG_HWA_GetUnlockStatus(WDOG_Type * pWdog)	
Description	Get WDOG unlock status, if locked, register can't be written.	
Diagram	N/A	
Parameters	Parameter	Description
	pWdog	Point to WDOG instance base register address.
Returns	bool. true as unlocked. false as locked.	

2.7.6.3 void WDOG_HWA_SetCounter(WDOG_Type * pWdog, uint32 u32Counter)

Function	void WDOG_HWA_SetCounter(WDOG_Type * pWdog, uint32 u32Counter)	
Description	This function Set COUNTER register value, for watchdog unlock and refresh usage.	
Diagram	 <pre> graph LR Wdg_LLD_Init --> Wdg_LLD_Unlock Wdg_LLD_Unlock --> WDOG_HWA_SetCounter Wdg_LLD_Trigger --> WDOG_HWA_SetCounter </pre>	
Parameters	Parameter	Description
	pWdog.	Point to WDOG instance base register address.
	u32Counter.	Configured register value
Returns	N/A	

2.7.6.4 void WDOG_HWA_SetCs(WDOG_Type * pWdog, uint32 u32Cs)

Function	void WDOG_HWA_SetCs(WDOG_Type * pWdog, uint32 u32Cs)	
Description	This function Set CS register value, for watchdog working mode configuration.	
Diagram	 <pre> graph LR Wdg_LLD_Init --> WDOG_HWA_SetCs </pre>	
Parameters	Parameter	Description
	pWdog	Point to WDOG instance base register address.
	u32Cs	Configured register value
Returns	N/A	

2.7.6.5 void WDOG_HWA_SetTimeout(WDOG_Type * pWdog, uint16 u16Timeout)

Function	void WDOG_HWA_SetTimeout(WDOG_Type * pWdog, uint16 u16Timeout)	
Description	Set TIMEOUT register value. for WDOG timeout value.	
Diagram	 <pre> graph LR Wdg_LLD_Init --> WDOG_HWA_SetTimeout </pre>	
Parameters	Parameter	Description

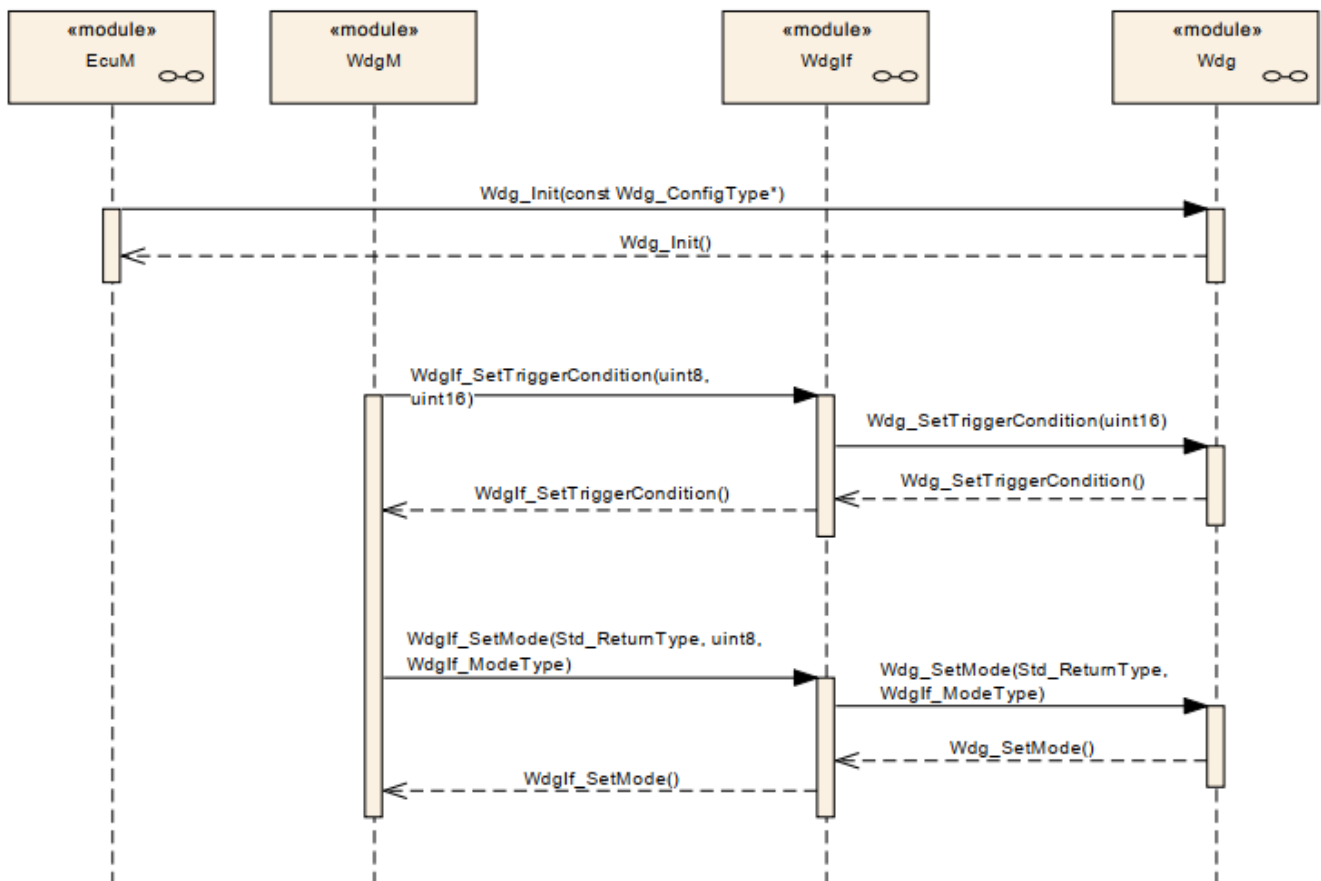
	pWdog	Point to WDOG instance base register address
	u16Timeout	Configured register value
Returns	N/A	

2.7.6.6 void WDOG_HWA_SetWindow(WDOG_Type * pWdog, uint16 u16Window)

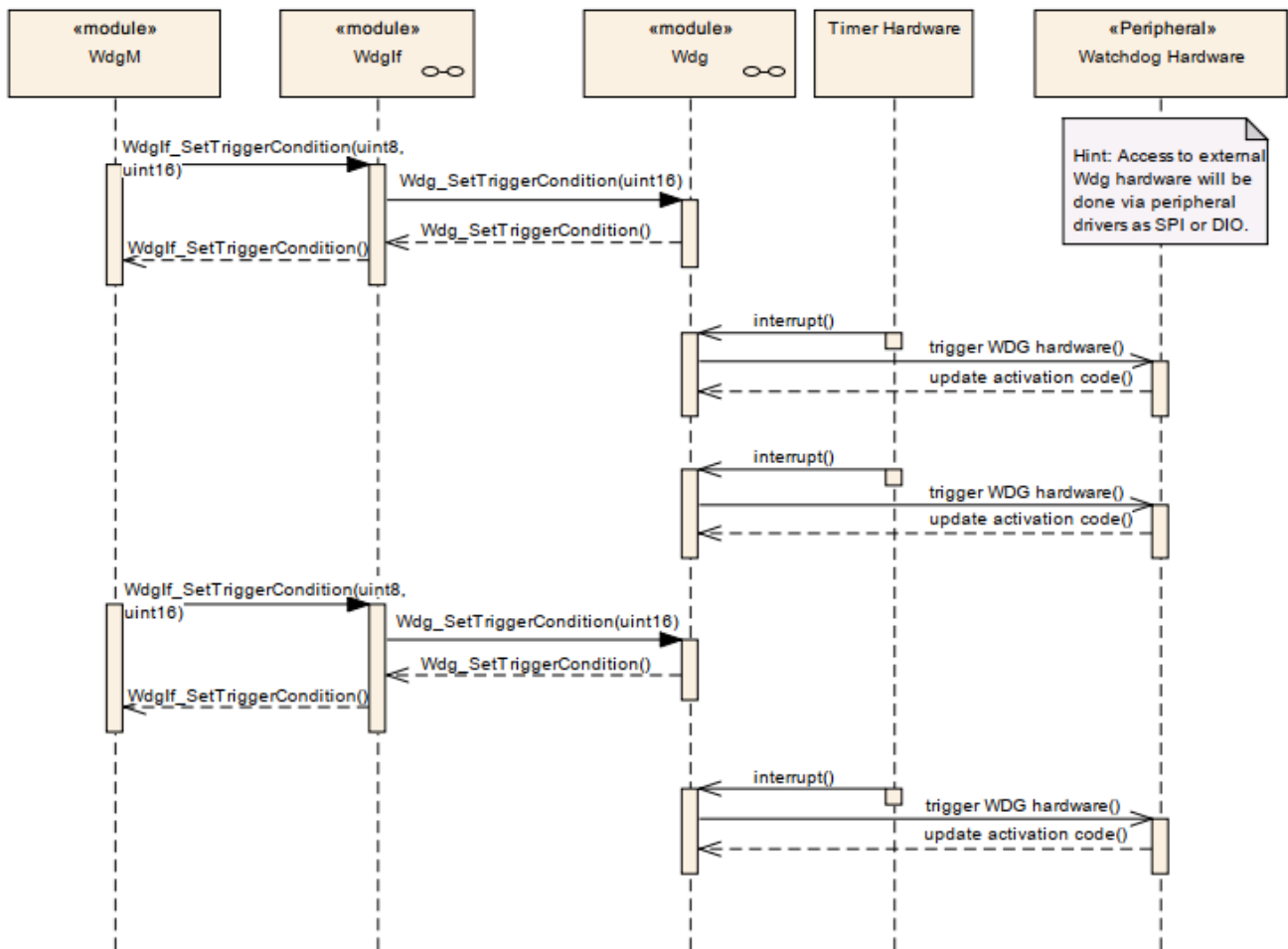
Function	void WDOG_HWA_SetWindow(WDOG_Type * pWdog, uint16 u16Window)	
Description	Set WINDOW register value, for windowed WDOG low threshold value.	
Diagram	 <pre> graph LR Wdg_LLD_Init[Wdg_LLD_Init] --> WDOG_HWA_SetWindow[WDOG_HWA_SetWindow] </pre>	
Parameters	Parameter	Description
	pWdog	Point to WDOG instance base register address
	u16Window	Configured register value
Returns	N/A	

2.8 API Sequence Diagram

2.8.1 Watchdog Initialization, Setting Trigger Condition and Mode



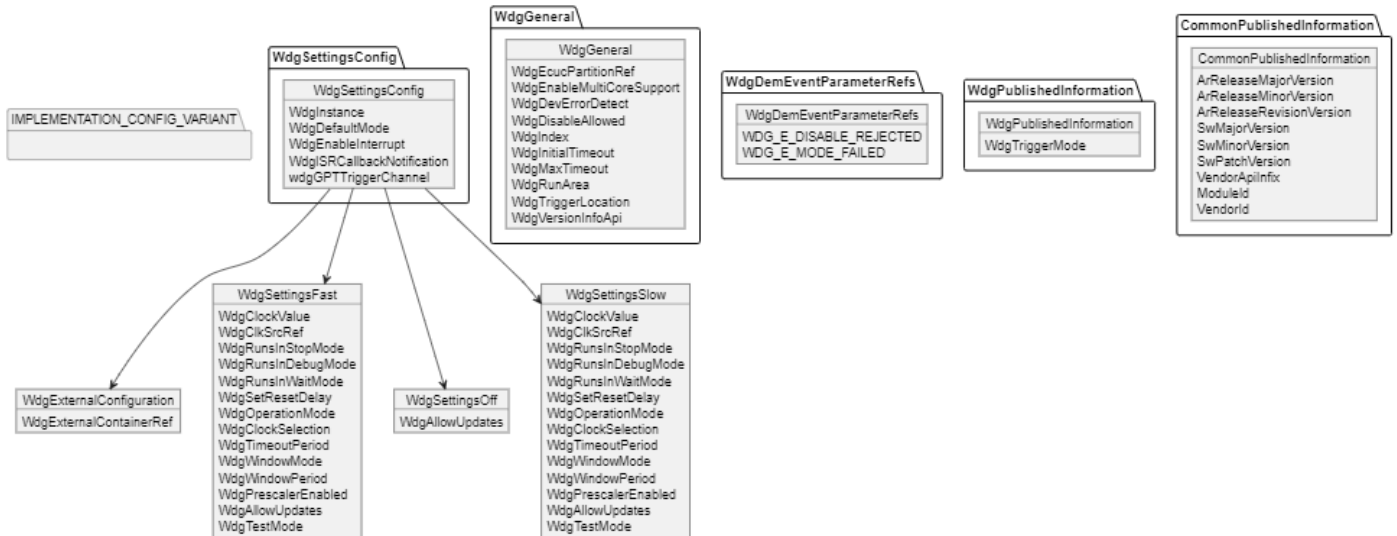
2.8.2 Data Exchange between Watchdog Driver and Hardware



Chapter 3 Tresos Configuration Items

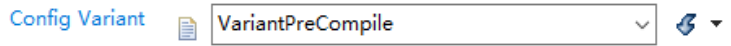
3.1 Container Inclusion Relation

The container inclusion relation is shown below:

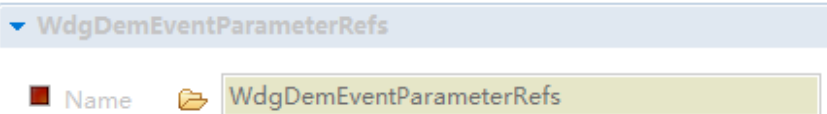


3.2 Containers and Variables

3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Config Variant
	Default	VariantPostBuild

3.2.2 WdgDemEventParameterRefs

Container	WdgDemEventParameterRefs	
Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId symbolic value. The standardized errors are provided in this container and can be extended by vendor-specific error references.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A

	Default	N/A
--	---------	-----

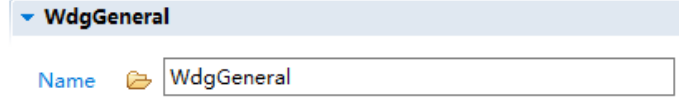
3.2.2.1 WDG_E_DISABLE_REJECTED

Container	WDG_E_DISABLE_REJECTED	
Description	Reference to the DemEventParameter which shall be issued when the error "Initialization or mode switch failed because it would disable the watchdog" has occurred.	
Screenshot		
Properties	Property	Value
	Type	SYMBOLIC-NAME-REFERENCE
	Label	N/A
	Default	N/A

3.2.2.2 WDG_E_MODE_FAILED

Container	WDG_E_MODE_FAILED	
Description	Reference to the DemEventParameter which shall be issued when the error "Setting a watchdog mode failed (during initialization or mode switch)" has occurred.	
Screenshot		
Properties	Property	Value
	Type	SYMBOLIC-NAME-REFERENCE
	Label	N/A
	Default	N/A

3.2.3 WdgGeneral

Container	WdgGeneral	
Description	All general parameters of the watchdog driver are collected here.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.3.1 WdgDevErrorDetect

Container	WdgDevErrorDetect	
Description	Compile switch to enable/disable development error detection for this module. - False: Development error detection disabled; - True: Development error detection enabled.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Development Error Detection

	Default	True
--	---------	------

3.2.3.2 WdgEcucPartitionRef

Container	WdgEcucPartitionRef	
Description	Maps the Wdg driver to zero or one ECUC partitions to make the modules API available in this partition.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Label	N/A
	Default	N/A

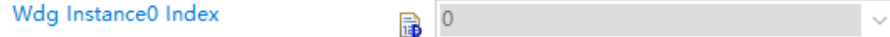
3.2.3.3 WdgEnableMultiCoreSupport

Container	WdgEnableMultiCoreSupport	
Description	When this parameter is enabled, the Wdg module will adapt to run in Multi-core. - False: Wdg module will not adapt to in Multi-core. - True: Wdg module will adapt to run in Multi-core.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Enable Multi-Core Support
	Default	False

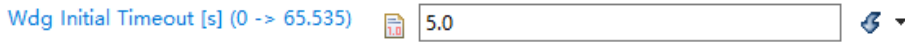
3.2.3.4 WdgDisableAllowed

Container	WdgDisableAllowed	
Description	Compile switch to allow/forbid disabling the watchdog driver during runtime. - False: Disabling the watchdog driver at runtime is not allowed; - True: Disabling the watchdog driver at runtime is allowed.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Disable Allowed
	Default	False

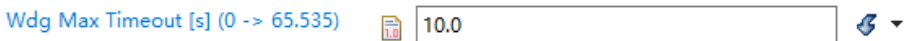
3.2.3.5 WdgIndex

Container	WdgIndex	
Description	Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Wdg Instance0 Index
	Default	0

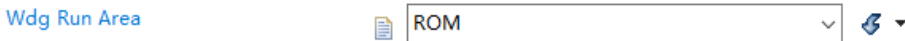
3.2.3.6 WdgInitialTimeout

Container	WdgInitialTimeout	
Description	The initial timeout (sec) for the trigger condition to be initialized during Init function. It shall be not larger than WdgMaxTimeout.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	Label	Wdg Initial Timeout [s]
	Default	0

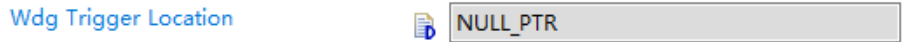
3.2.3.7 WdgMaxTimeout

Container	WdgMaxTimeout	
Description	The maximum timeout (seconds) to which the watchdog trigger condition can be initialized.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	Label	Wdg Max Timeout [s]
	Default	0

3.2.3.8 WdgRunArea

Container	WdgRunArea	
Description	Represents the watchdog driver execution area is either from ROM(Flash) or RAM as required with the particular microcontroller.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Run Area
	Default	ROM

3.2.3.9 WdgTriggerLocation

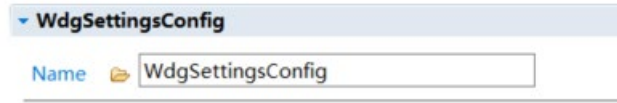
Container	WdgTriggerLocation	
Description	Location (memory address) of the watchdog trigger routine. Note: Not supported by the current hardware.	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Label	Wdg Trigger Location
	Default	NULL_PTR

3.2.3.10 WdgVersionInfoApi

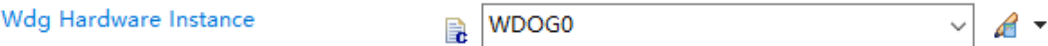
Container	WdgVersionInfoApi
------------------	-------------------

Description	Compile switch to enable or disable the version information API. - True: API enabled - False: API disabled	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Provide Version Info API
	Default	True

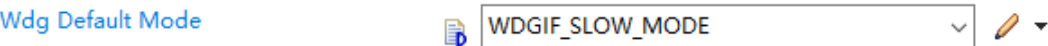
3.2.4 WdgSettingsConfig

Container	WdgSettingsConfig	
Description	Configuration items for the different watchdog settings, including those for external watchdog hardware.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.4.1 WdgInstance

Container	WdgInstance	
Description	Select specific hardware instance for watchdog driver initialization.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Hardware Instance
	Default	WDG0

3.2.4.2 WdgDefaultMode

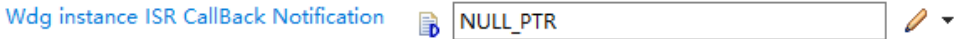
Container	WdgDefaultMode	
Description	Default mode for watchdog driver initialization.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Default Mode
	Default	WDGIF_SLOW_MODE

3.2.4.3 WdgEnableInterruptFunction

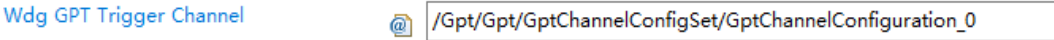
Container	WdgEnableInterruptFunction	
Description	Compile switch to enable/disable watchdog instance interrupt function.	

	- True: Enable watchdog instance interrupt function; - False: Disable watchdog instance interrupt function.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Enable WDG interrupt function
	Default	False

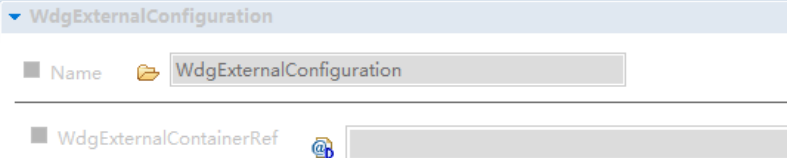
3.2.4.4 WdgISRCallbackNotification

Container	WdgISRCallbackNotification	
Description	Callback notification for the watchdog instance ISR function.	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Label	Wdg instance ISR Callback Notification
	Default	NULL_PTR

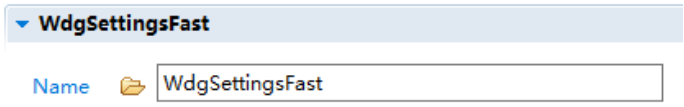
3.2.4.5 wdgGPTTriggerChannel

Container	wdgGPTTriggerChannel	
Description	Reference to the GptChannel configuration which set for the watchdog servicing routine implementation.	
Screenshot		
Properties	Property	Value
	Type	CHOICE-REFERENCE
	Label	Wdg GPT Trigger Channel
	Default	N/A

3.2.4.6 WdgExternalContainerRef

Container	WdgExternalContainerRef	
Description	a DioChannelGroup container in case the hardware watchdog is connected via DIO pins a SpiSequenceConfiguration container in case the watchdog hardware is accessed via SPI Note: This parameter is not used by current implementation	
Screenshot		
Properties	Property	Value
	Type	CHOICE-REFERENCE
	Label	N/A
	Default	N/A

3.2.4.7 WdgSettingsFast

Container	WdgSettingsFast	
Description	Hardware dependent settings for the watchdog driver's fast mode.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.4.7.1 WdgClockValue

Container	WdgClockValue	
Description	This is the Implementation Specific parameter. Indicates WDG Clock Value in KHz.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Wdg Clock Value[KHz]
	Default	XPATH

3.2.4.7.2 WdgClkSrcRef

Container	WdgClkSrcRef	
Description	Reference to the WdgClockReferencePoint from which the clock is derived.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Label	N/A
	Default	N/A

3.2.4.7.3 WdgRunsInStopMode

Container	WdgRunsInStopMode	
Description	This is the Implementation Specific parameter. - Enabled: WDOG continues to count even while the processor core is in stop mode. - Disabled: WDOG stops counting if the processor core is in stop mode.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Runs in Stop mode
	Default	True

3.2.4.7.4 WdgRunsInDebugMode

Container	WdgRunsInDebugMode
------------------	--------------------

Description	This is the Implementation Specific parameter. - Enabled: WDOG continues to count even while the device enters the debug mode. - Disabled: WDOG stops counting if the processor core when the device enters the debug mode	
Screenshot	Wdg Runs In Debug Mode 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Runs in Debug Mode
	Default	True

3.2.4.7.5 WdgRunsInWaitMode

Container	WdgRunsInWaitMode	
Description	This is the Implementation Specific parameter. - Enabled: WDOG continues to count even while the device enters the wait mode. - Disabled: WDOG stops counting if the processor core when the device enters the wait mode	
Screenshot	Wdg Runs In Wait Mode 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Runs in Wait Mode
	Default	False

3.2.4.7.6 WdgSetResetDelay

Container	WdgSetResetDelay	
Description	This is the Implementation Specific parameter. Set the value of the WDOG reset delay when enabling the WDOG interrupt. The default value is 128 bus clock cycles. • WDG_RST_DLY_128: Delay of 128 bus clocks to generate the reset. • WDG_RST_DLY_256: Delay of 256 bus clocks to generate the reset. • WDG_RST_DLY_512: Delay of 512 bus clocks to generate the reset. • WDG_RST_DLY_1024: Delay of 1024 bus clocks to generate the reset.	
Screenshot	Wdg Set Reset Delay 	
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Set Reset Delay
	Default	WDG_RST_DLY_128

3.2.4.7.7 WdgOperationMode

Container	WdgOperationMode	
Description	This is the Implementation Specific parameter. - ResetOnTimeOut: Generate a reset on a time-out. - Interrupt: Generate an interrupt on an initial time-out, reset after 128 cycles.	
Screenshot	Wdg Operation Mode 	
Properties	Property	Value

	Type	ENUMERATION
	Label	Wdg Operation Mode
	Default	ResetOnTimeOut

3.2.4.7.8 WdgClockSelection

Container	WdgClockSelection	
Description	Selection for WDOG clock.	
Screenshot	Wdg Clock Selection 	
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Clock Selection
	Default	Bus_Clock

3.2.4.7.9 WdgTimeoutPeriod

Container	WdgTimeoutPeriod	
Description	Software Watchdog Time-Out Period in seconds. Selects the time-out period for the WDOG.	
Screenshot	Wdg Timeout Period [s] (0 -> 16776.96) 	
Properties	Property	Value
	Type	FLOAT
	Label	WdgTimeout Period [s]
	Default	0.01

3.2.4.7.10 WdgWindowMode

Container	WdgWindowMode	
Description	- Disabled: Regular mode, service sequence can be done at any time. - Enabled: Windowed mode, the service sequence is only valid when the up counter is higher than the value in the WDOG WIN register.	
Screenshot	Wdg Window Mode 	
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Window Mode
	Default	False

3.2.4.7.11 WdgWindowPeriod

Container	WdgWindowPeriod	
Description	This is the Implementation Specific parameter. Window start value. When window mode is enabled, the service sequence can only be written when the internal up counter is higher than this value.	
Screenshot	Wdg Window Period[s] (0 -> 16776.96) 	
Properties	Property	Value
	Type	FLOAT

	Label	Wdg Window Period[s]
	Default	0.004

3.2.4.7.12 WdgPrescalerEnabled

Container	WdgPrescalerEnabled	
Description	Switch to enable/disable prescaler function. - Disabled: Prescaler Disabled. - Enabled: Prescaler Enabled.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Prescaler Enabled
	Default	True

3.2.4.7.13 WdgAllowUpdates

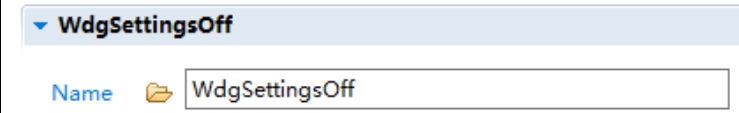
Container	WdgAllowUpdates	
Description	Whether to allow the software to reconfigure the watchdog without a reset. Disabled: Updates not allowed. Enabled: Updates allowed.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Allow Updates
	Default	True

3.2.4.7.14 WdgTestMode

Container	WdgTestMode	
Description	TestModeDisabled: Watchdog test mode disabled. UserModeEnabled: Watchdog user mode enabled. TestModeLowByte: Watchdog test mode enabled, only the low byte is used. TestModeHighByte: Watchdog test mode enabled, only the high byte is used.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	WdgTestMode
	Default	TestModeDisabled

3.2.4.8 WdgSettingsOff

Container	WdgSettingsOff	
Description	Hardware dependent settings for the watchdog driver's off mode.	

Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

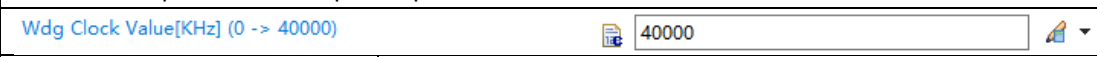
3.2.4.8.1 WdgAllowUpdates

Container	WdgAllowUpdates	
Description	Wdg Allow Updates. - Disabled: Updates not allowed. - Enabled: Updates allowed.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Allow Updates
	Default	True

3.2.4.9 WdgSettingsSlow

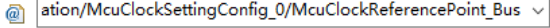
Container	WdgSettingsSlow	
Description	Hardware dependent settings for the watchdog driver's slow mode.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE
	Label	N/A
	Default	N/A

3.2.4.9.1 WdgClockValue

Container	WdgClockValue	
Description	This is the Implementation Specific parameter. Indicates WDG Clock Value in KHz.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Label	Wdg Clock Value[KHz]
	Default	XPATH

3.2.4.9.2 WdgClkSrcRef

Container	WdgClkSrcRef	
Description	Reference to the WdgClockReferencePoint from which the clock is derived.	

Screenshot	 WdgClkSrcRef  ation/McuClockSettingConfig_0/McuClockReferencePoint_Bus	
Properties	Property	Value
	Type	REFERENCE
	Label	N/A
	Default	N/A

3.2.4.9.3 WdgRunsInStopMode

Container	WdgRunsInStopMode	
Description	This is the Implementation Specific parameter. - Enabled: WDOG continues to count even while the processor core is in stop mode. - Disabled: WDOG stops counting if the processor core is in stop mode.	
Screenshot	   Wdg Runs In Stop mode	
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Runs In Stop mode
	Default	True

3.2.4.9.4 WdgRunsInDebugMode

Container	WdgRunsInDebugMode	
Description	This is the Implementation Specific parameter. - Enabled: WDOG continues to count even while the device enters the debug mode. - Disabled: WDOG stops counting if the processor core when the device enters the debug mode.	
Screenshot	   Wdg Runs In Debug Mode	
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Runs In Debug Mode
	Default	True

3.2.4.9.5 WdgRunsInWaitMode

Container	WdgRunsInWaitMode	
Description	This is the Implementation Specific parameter. - Enabled: WDOG continues to count even while the device enters the wait mode. - Disabled: WDOG stops counting if the processor core when the device enters the wait mode.	
Screenshot	   Wdg Runs In Wait Mode	
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Runs In Wait Mode
	Default	False

3.2.4.9.6 WdgSetResetDelay

Container	WdgSetResetDelay
------------------	------------------

Description	This is the Implementation Specific parameter. Set the value of the WDOG reset delay when enabling the WDOG interrupt. The default value is 128 bus clock cycles. • WDG_RST_DLY_128: Delay of 128 bus clocks to generate the reset. • WDG_RST_DLY_256: Delay of 256 bus clocks to generate the reset. • WDG_RST_DLY_512: Delay of 512 bus clocks to generate the reset. • WDG_RST_DLY_1024: Delay of 1024 bus clocks to generate the reset.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Set Reset Delay
	Default	WDG_RST_DLY_128

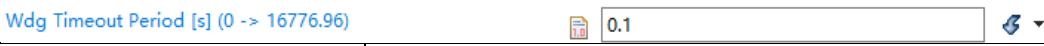
3.2.4.9.7 WdgOperationMode

Container	WdgOperationMode	
Description	This is the Implementation Specific parameter. - ResetOnTimeOut: Generate a reset on a time-out. - Interrupt: Generate an interrupt on an initial time-out, reset after 128 cycles.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Operation Mode
	Default	ResetOnTimeOut

3.2.4.9.8 WdgClockSelection

Container	WdgClockSelection	
Description	Select clock source of WDOG.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Wdg Clock Selection
	Default	Bus_Clock

3.2.4.9.9 WdgTimeoutPeriod

Container	WdgTimeoutPeriod	
Description	Software Watchdog Time-Out Period in seconds. Selects the time-out period for the WDOG.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	Label	Wdg Timeout Period [s]
	Default	0.01

3.2.4.9.10 WdgWindowMode

Container	WdgWindowMode	
Description	- Disabled: Regular mode, service sequence can be done at any time. - Enabled: Windowed mode, the service sequence is only valid when the up counter is higher than value in the WDOG WIN register.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Window Mode
	Default	False

3.2.4.9.11 WdgWindowPeriod

Container	WdgWindowPeriod	
Description	This is the Implementation Specific parameter. Window start value. When window mode is enabled, the service sequence can only be written when the internal up counter is higher than this value.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	Label	Wdg Window Period[s]
	Default	0.004

3.2.4.9.12 WdgPrescalerEnabled

Container	WdgPrescalerEnabled	
Description	Switch to enable/disable prescaler function. - Disabled: Prescaler Disabled. - Enabled: Prescaler Enabled.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Prescaler Enabled
	Default	True

3.2.4.9.13 WdgAllowUpdates

Container	WdgAllowUpdates	
Description	Whether to allow the software to reconfigure the watchdog without a reset. - Disabled: Updates not allowed. - Enabled: Updates allowed.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Label	Wdg Allow Updates

	Default	True
--	---------	------

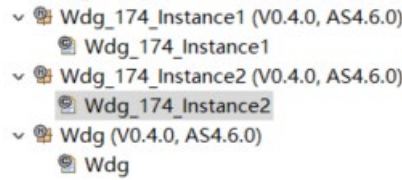
3.2.4.9.14 WdgTestMode

Container	WdgTestMode	
Description	TestModeDisabled: Watchdog test mode disabled. UserModeEnabled: Watchdog user mode enabled. TestModeLowByte: Watchdog test mode enabled, only the low byte is used. TestModeHighByte: Watchdog test mode enabled, only the high byte is used.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	WdgTestMode
	Default	TestModeDisabled

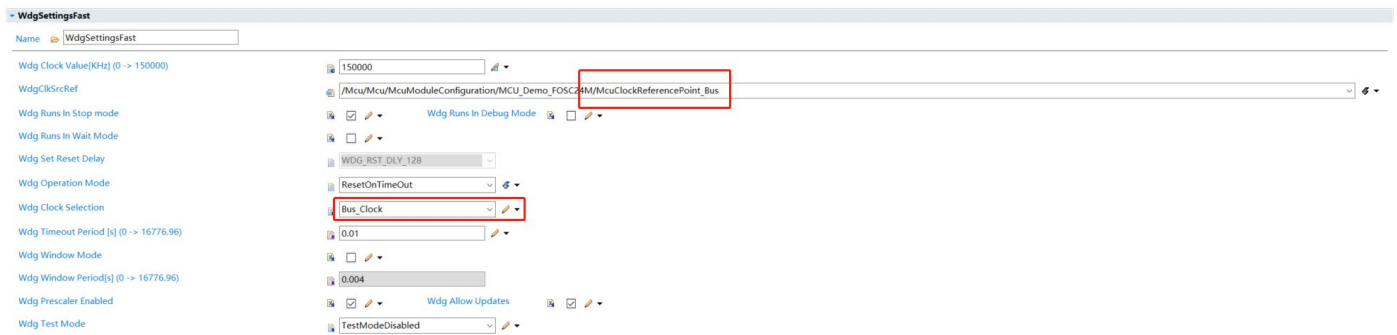
Chapter 4 Configuration Guides

4.1 Configuration Item Constraint

- 1) In WDG module, each watchdog instance has a different module.



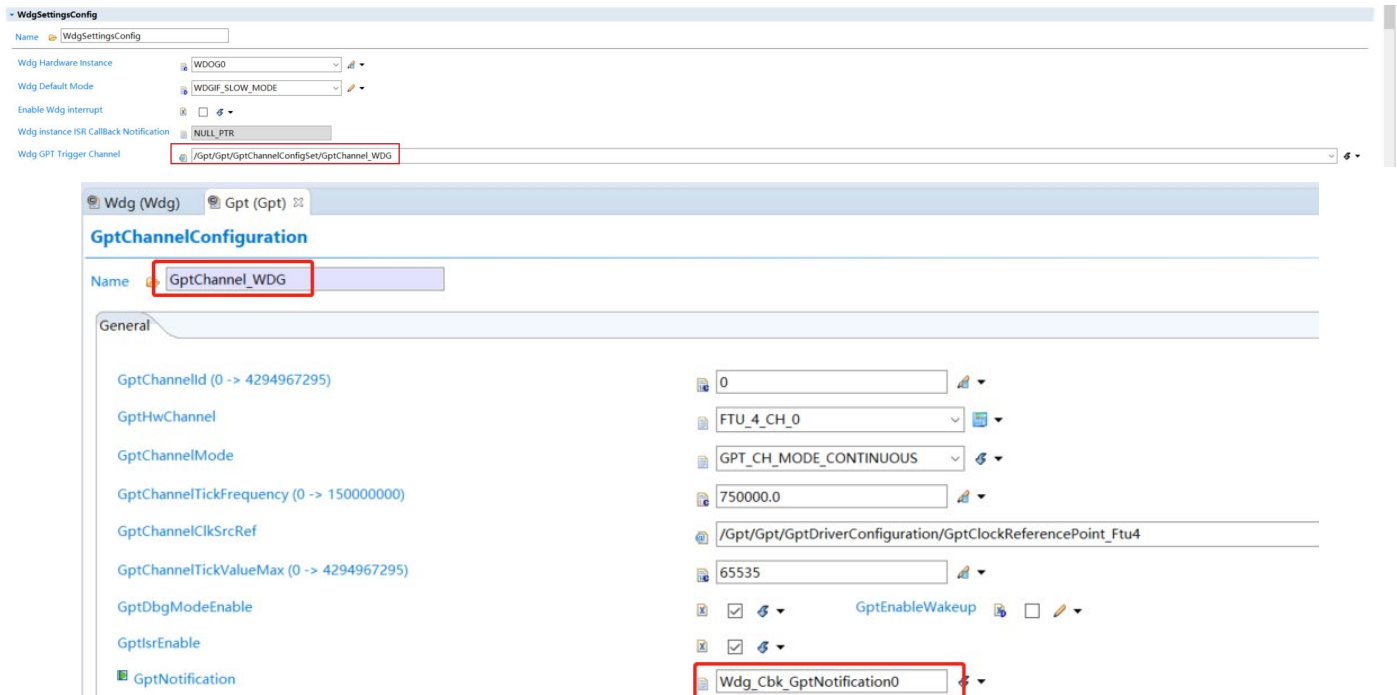
- 2) In WDG setting, the WDG clock selection should be matched with wdgClkSrcRef.



- 3) In WDG module, the user should select the GPT resource to serve the watchdog in GPT ISR. The user should configure the GPT notification properly.

Wdg0 -- Wdg_Cbk_GptNotification0

Wdg1 -- Wdg_Cbk_GptNotification1



4.2 WDG Usage Common Steps

Basically, the WDG module can be configured by following the below 4 steps:

- 1) Configure WDG General.
- 2) Add the related watchdog configuration setting in WdgSettingsConfig.
- 3) Choose which API will be used.
- 4) Generate configuration files.

4.3 WDG Configuration Demo

The following tables represent the configuration data shown in the screenshots:

WdgGeneral Configuration

Property	Value
Config Variant	VariantPreCompile
WdgDemEventParameterRefs	WdgDemEventParameterRefs
WDG_DISABLE_REJECTED	/Dem/DemConfigSet/DemEventParameter_0
WDG_E_MODE_FAILED	/Dem/DemConfigSet/DemEventParameter_0
WdgEcuPartitionRef	/EcuC/EcuC/EcuPartitionCollection_0/EcuPartition_0
Wdg Enable Multi-Core Support	☒ Development Error Detection
Wdg Disable Allowed	☒
Wdg Instance0 Index (0 -> 255)	0
Wdg Initial Timeout [s] (0 -> 65.535)	0.03
Wdg Max Timeout [s] (0 -> 65.535)	0.1
Wdg Run Area	ROM
Wdg Trigger Location	NULL_PTR
Provide Version Info API	☒

WdgSettingsConfig Configuration

Property	Value
Name	WdgSettingsConfig
Wdg Hardware Instance	WDG0
Wdg Default Mode	WDGIF_SLOW_MODE
Enable Wdg Interrupt	☐
Wdg Instance ISR CallBack Notification	NULL_PTR
Wdg GPT Trigger Channel	/Gpt/Gpt/GptChannelConfigSet/GptChannel_WDG
WdgExternalConfiguration	WdgExternalConfiguration
WdgExternalContainerRef	

WdgSettingsFast Configuration

Property	Value
Name	WdgSettingsFast
Wdg Clock Value[KHz] (0 -> 150000)	24000
WdgCkSrcRef	/Mcu/Mcu/McuModuleConfiguration/MCU_Demo_FOSC24M/McuClockReferencePoint_Fosc
Wdg Runs In Stop mode	☒ Wdg Runs In Debug Mode
Wdg Runs In Wait Mode	☐
Wdg Set Reset Delay	WDG_RST_DLY_128
Wdg Operation Mode	ResetOnTimeOut
Wdg Clock Selection	FOSC_Clock
Wdg Timeout Period [s] (0 -> 16776.96)	0.01
Wdg Window Mode	☐
Wdg Window Period[s] (0 -> 16776.96)	0.004
Wdg Prescaler Enabled	☒ Wdg Allow Updates
Wdg Test Mode	TestModeDisabled

WdgSettingsOff

Name

WdgSettingsOff

Wdg Allow Updates

WdgSettingsSlow

Name

WdgSettingsSlow

Wdg Clock Value[KHz] (0 -> 150000)

24000

Wdg ClkSrcRef

/Mcu/McuModuleConfiguration/MCU_Demo_FOSC24M/McuClockReferencePoint_Fosc

Wdg Runs In Stop mode

☒

Wdg Runs In Debug Mode

Wdg Runs In Wait Mode

☐

Wdg Set Reset Delay

WDG_RST_DLY_128

Wdg Operation Mode

ResetOnTimeOut

Wdg Clock Selection

FOSC_Clock

Wdg Timeout Period [s] (0 -> 16776.96)

0.1

Wdg Window Mode

☐

Wdg Window Period[s] (0 -> 16776.96)

0.004

Wdg Prescaler Enabled

☒

Wdg Allow Updates

Wdg Test Mode

TestModeDisabled

WdgPublishedInformation

Name

WdgPublishedInformation

Wdg Trigger Mode

WDG_BOTH